

Design Patterns – Master Section Preparation Notes

This is the **core set I would prepare before the Infosys interview**. I am basing this on your uploaded [Design-Patterns\(1\).pdf](#), especially its first 20 questions and the later comparison/application questions. The document itself emphasizes Creational, Structural, Behavioral patterns, SOLID, IoC/DI, and practical pattern selection.

The goal is not to memorize definitions. For every major topic, be ready to explain:

Problem → Idea → Structure → Example → When to use → Limitation

1. Design Patterns Fundamentals

What is a Design Pattern?

A design pattern is a **reusable solution to a recurring software design problem**.

It is not a ready-made piece of code. It is a **design approach/blueprint** that tells us how classes and objects can be organized.

Why use Design Patterns?

They help with:

- loose coupling
- maintainability
- reusability
- scalability
- communication between developers
- avoiding repeated design mistakes

Your PDF describes patterns as reusable, proven solutions that improve flexibility, maintainability and communication.

Interview answer

"A design pattern is a proven, reusable approach for solving a recurring software design problem. It provides a common vocabulary and helps us build loosely coupled, maintainable and extensible systems."

2. Three Categories

Your PDF classifies patterns into three families.

Category	Main purpose	Must-know examples
Creational	Object creation	Singleton, Factory, Builder
Structural	Object/class composition	Adapter, Decorator, Facade
Behavioral	Object communication/behavior	Strategy, Observer, Command

Easy memory trick

Creational = How is an object created?

Structural = How are objects connected?

Behavioral = How do objects communicate/behave?

3. SOLID Principles

This is one of the most important sections because your Infosys guideline specifically mentions SOLID.

Your PDF covers all five principles and connects them with patterns such as Strategy, Factory and Dependency Injection.

S – Single Responsibility Principle

A class should have **one reason to change**.

Bad:

```
Employee
├── calculateSalary()
├── saveToDatabase()
└── generateReport()
```

This class has multiple responsibilities.

Better:

```
Employee
SalaryCalculator
EmployeeRepository
ReportGenerator
```

Interview keyword

One class = one responsibility / one reason to change.

0 – Open/Closed Principle

Software entities should be **open for extension but closed for modification**.

Suppose:

```
PaymentService
```

supports:

```
UPI
Card
Cash
```

Instead of modifying the existing service every time a new payment method appears, create new implementations.

Pattern strongly associated with OCP

Strategy

Your PDF explicitly describes Strategy, Decorator, Factory and Observer as patterns that can support OCP.

Interview question

"Why is Strategy a good example of OCP?"

Because adding a new strategy can happen through a new class without changing the context.

L – Liskov Substitution Principle

A child object should be usable wherever its parent is expected without breaking correctness.

Classic conceptual example:

```
Bird
└─ FlyingBird
```

If `Bird` exposes `fly()`, then a non-flying bird can violate the expected contract.

Interview focus

Don't just memorize the sentence. Understand:

Inheritance should preserve the contract of the base abstraction.

I – Interface Segregation Principle

Clients should not be forced to depend on methods they do not use.

Bad:

```
Worker
├─ work()
├─ eat()
└─ sleep()
```

Suppose a robot worker doesn't need `eat()` or `sleep()`.

Split interfaces:

```
Workable  
Eatable  
Sleepable
```

Keyword

Small, focused interfaces instead of one huge interface.

D – Dependency Inversion Principle

High-level modules should depend on **abstractions**, not concrete implementations.

Bad:

```
class PaymentService {  
    CreditCardProcessor processor;  
};
```

Better:

```
class PaymentProcessor {  
public:  
    virtual void pay() = 0;  
};  
  
class PaymentService {  
    PaymentProcessor* processor;  
};
```

Now the high-level class depends on the abstraction.

Your PDF describes DIP as reversing dependency flow so high-level and low-level modules communicate through abstractions.

4. Singleton Pattern

This is **explicitly relevant to your Infosys interview**, so know it very well.

Definition

Singleton ensures that:

1. only one instance of a class exists
2. there is a global access point to that instance

Your PDF gives the standard components as private constructor, static instance and public `getInstance()` .

Basic structure

```
class Singleton {  
private:  
    Singleton() {}  
  
public:  
    static Singleton& getInstance() {  
        static Singleton instance;  
        return instance;  
    }  
};
```

Why private constructor?

To prevent:

```
Singleton a;  
Singleton b;
```

from creating multiple instances.

Common uses

Your PDF gives examples such as:

- configuration manager

- logging framework
 - shared resource manager
 - Java `Runtime`
-

Singleton disadvantages

This is very important.

- global state
- tight coupling
- harder testing/mock
- thread-safety concerns
- can become overused

The PDF explicitly highlights these drawbacks.

Interview question

"Why can Singleton be considered bad design?"

Answer:

"Although Singleton guarantees a single instance, it introduces global state and tight coupling. That makes dependencies less explicit and can make unit testing harder because replacing the global instance with a mock becomes difficult."

Thread-safe Singleton

Know the idea of:

- eager initialization
- lazy initialization
- synchronized method
- double-checked locking
- enum-based Singleton

Your PDF specifically covers these implementation approaches.

Must understand

Lazy initialization

Object is created only when first requested.

Thread safety

Two threads should not accidentally create two instances.

5. Factory Method

Another **must-master** topic because it is directly mentioned in the Infosys preparation guidelines.

Problem

Suppose client code does:

```
Circle c;  
Rectangle r;  
Triangle t;
```

Now the client knows all concrete classes.

Factory moves object creation away from the client.

```
Client  
  |  
  v  
Factory  
 /  |  \  
Circle Rectangle Triangle
```

Definition

Factory Method defines an interface for creating an object while allowing subclasses to determine the concrete product.

Your PDF describes this as delegating instantiation to subclasses.

6. Factory vs Abstract Factory

This is a **must-memorize comparison**.

Factory Method

Creates **one product type through a method**.

```
createShape()  
  ↓  
Circle / Rectangle
```

Abstract Factory

Creates a **family of related products**.

```
WindowsFactory  
├── WindowsButton  
└── WindowsMenu  
  
MacFactory  
├── MacButton  
└── MacMenu
```

Your PDF explicitly states:

Factory Method = one product
Abstract Factory = product family.

Interview question

"When would you choose Abstract Factory?"

When multiple related products need to be created together while maintaining compatibility between them.

7. Strategy Pattern

This is another major pattern.

Problem

Suppose:

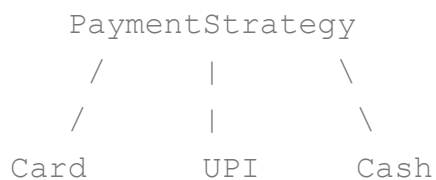
```
PaymentService
```

contains:

```
if (card)
    ...
else if (upi)
    ...
else if (cash)
    ...
```

As payment methods grow, the class becomes difficult to maintain.

Strategy separates algorithms/behaviors.



The Context holds a Strategy and delegates behavior to it.

Your PDF explicitly describes Strategy as encapsulating interchangeable algorithms and using composition for runtime switching.

Strategy + OCP

Very important connection:

```
Context
|
Strategy interface
```



Adding D:

```
Strategy D
```

doesn't require changing Context.

Therefore:

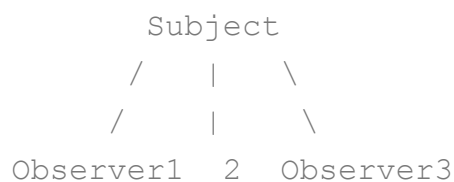
Strategy supports OCP.

8. Observer Pattern

Definition

Observer creates a **one-to-many relationship**.

One Subject changes → many Observers get notified.



Your PDF describes Subject, Observer interface and concrete observers as the key components.

Example

Stock price system:

```
StockPrice
|
+---- MobileApp
+---- WebDashboard
+---- TradingScreen
```

When stock price changes:

```
notify()
```

all observers update.

Typical interview examples

- notifications
 - event handling
 - GUI systems
 - MVC
-

9. Builder Pattern

Problem

Suppose an object has:

```
name
age
address
phone
email
country
...
```

A constructor becomes difficult:

```
User(name, age, address, phone, email, country, ...);
```

Builder solves this through step-by-step construction.

Your PDF specifically identifies constructor telescoping as the problem and describes fluent APIs and immutable construction.

Example

```
User user =  
    UserBuilder("RK")  
        .age(22)  
        .address("Chennai")  
        .email("x@y.com")  
        .build();
```

Remember

Builder = complex object construction

10. Adapter Pattern

Problem

Two classes have incompatible interfaces.

```
Client  
|  
Target Interface  
|  
Adapter  
|  
Adaptee
```

The Adapter translates one interface into another.

Your PDF describes Adapter primarily for legacy/third-party integration.

Real-world analogy

USB-C laptop + USB-A device:

```
Laptop  
↓  
Adapter  
↓  
USB-A device
```

Interview sentence

"Adapter allows an existing incompatible class to work with the interface expected by the client without modifying the existing class."

11. Decorator Pattern

Definition

Decorator adds behavior to an existing object **without modifying its original class**.

```
Coffee
  ↓
MilkDecorator
  ↓
SugarDecorator
```

Each wrapper adds functionality.

Your PDF gives the text-editor example and explains runtime extension through wrapping.

Key idea

Decorator = add responsibility

12. Facade Pattern

Problem

A subsystem contains many classes and complex interactions.

Instead of:

```
client → projector  
client → sound system  
client → lights  
client → player
```

we provide:

```
client  
  ↓  
Facade  
  ↓  
multiple subsystems
```

Your PDF describes Facade as a unified interface hiding subsystem complexity.

Interview sentence

"Facade provides a simple high-level interface to a complex subsystem."

13. Command Pattern

Definition

Command converts a request/action into an object.

```
Invoker  
  |  
Command  
  |  
Receiver
```

Your PDF identifies the main components as Command, Concrete Command, Invoker and Receiver, and discusses undo/redo using command history.

Example

Text editor:

```
TypeCommand  
DeleteCommand  
PasteCommand
```

Store commands:

```
Command History Stack
```

Then:

```
undo()  
redo()
```

becomes easier.

Key idea

Command = encapsulate an action/request

14. IoC and Dependency Injection

This is easy to confuse with DIP.

IoC

Inversion of Control means control over object creation/lifecycle/dependencies is moved outside the class, often to a framework/container.

Your PDF uses Spring as the example.

Instead of:

```
class A {  
    B b = new B();  
}
```

we can have:

```
class A {  
    B b;  
  
    A(B b) {  
        this.b = b;  
    }  
}
```

An external mechanism supplies **B**.

Dependency Injection

DI is the technique of supplying dependencies from outside.

Main types

Constructor injection

```
A(B b)
```

Setter injection

```
setB(B b)
```

Your PDF specifically describes constructor, setter and interface injection.

15. DIP vs DI vs IoC

This is a very common conceptual trap.

Concept	Meaning
DIP	Design principle
DI	Technique for supplying dependencies
IoC	Broader concept of transferring control

Easy memory

DIP → "Depend on abstraction"

DI → "Give me the dependency"

IoC → "I don't control the creation; external system does"

16. Critical Comparisons

These are worth revising separately.

Factory vs Builder

Factory → Which object?

Builder → How to construct it?

Your PDF explicitly describes Factory as one-step/polymorphic creation and Builder as multi-step configuration.

Factory Method vs Abstract Factory

Factory Method → one product

Abstract Factory → family of related products

Strategy vs State

Strategy:

Client selects the behavior.

State:

Object changes behavior based on internal state.

Your PDF explicitly makes this distinction.

Strategy vs Template Method

Strategy → composition

Template Method → inheritance

Strategy can swap the whole algorithm; Template Method fixes the algorithm skeleton and lets subclasses vary steps.

Adapter vs Decorator

Adapter → changes interface

Decorator → adds behavior

Adapter vs Facade

Adapter → compatibility

Facade → simplification

Your PDF explicitly makes this distinction later in the document.

Decorator vs Proxy

Decorator → add behavior

Proxy → control access

Command vs Strategy

Command → "Do this action"

Strategy → "Use this algorithm"

17. Composition vs Inheritance

This is an important OOP/design principle behind many patterns.

Inheritance

`Dog IS-A Animal`

Composition

`Car HAS-A Engine`

Patterns such as Strategy and Decorator heavily use composition.

Your PDF explicitly contrasts composition as more flexible with inheritance as creating more rigid hierarchies, and identifies Strategy/Decorator as composition-oriented.

Interview sentence

"Prefer composition when behavior should be replaceable or changeable independently; use inheritance when there is a genuine stable is-a relationship."

18. The "Why?" Questions You Should Practice

After studying the above, be able to answer these without notes:

Singleton

Why private constructor?

Why can Singleton hurt testing?

How do you make Singleton thread-safe?

Factory

Why not directly use `new`?

Factory vs Abstract Factory?

Strategy

Why is Strategy related to OCP?

Why composition instead of inheritance?

Observer

How does one-to-many notification work?

Builder

What is constructor telescoping?

Adapter

Adapter vs Bridge?

Decorator

Why not simply use inheritance to add functionality?

Facade

Does Facade change the underlying subsystem?

Command

Why is Command useful for undo/redo?

DI

DIP vs DI?

DI vs Singleton?

19. One-page revision map

Memorize this before the interview:

DESIGN PATTERNS

- |
- ├─ CREATIONAL
 - | └─ Singleton → one instance
 - | └─ Factory → create/select object
 - | └─ Abstract Factory → family of objects
 - | └─ Builder → complex step-by-step object
- |
- ├─ STRUCTURAL
 - | └─ Adapter → incompatible interfaces
 - | └─ Decorator → add behavior
 - | └─ Facade → simplify subsystem
- |
- └─ BEHAVIORAL
 - | └─ Strategy → interchangeable algorithms
 - | └─ Observer → one-to-many notification
 - | └─ Command → action/request as object

SOLID

- |
- ├─ S → one responsibility
- ├─ O → extend, don't modify
- ├─ L → subtype substitution
- ├─ I → focused interfaces
- └─ D → depend on abstractions

DEPENDENCY CONCEPTS

- |
- ├─ DIP → principle
- ├─ DI → technique
- └─ IoC → external control

KEY RELATIONSHIPS

- |
- ├─ Strategy → OCP
- ├─ Factory → DIP
- ├─ DI → DIP
- ├─ Decorator → OCP
- ├─ Composition → Strategy/Decorator
- └─ Command → Undo/Redo

What I would do tonight

Do **not** reread all 60 pages.

Study in this order:

**SOLID → Singleton → Factory → Strategy → Observer → Builder → Adapter → Decorator → Facade
→ Command → IoC/DI → comparisons**

Then close the PDF and explain each aloud in **30–60 seconds**. For Singleton, Factory and Strategy, write a small implementation from memory.

That will prepare you much better for an interview discussion than memorizing all the remaining patterns.